

Comprehensive Service Enumeration and Exploitation Report: Target 10.129.62.65

Step 1: Initial Reconnaissance (Port Scanning)

Topic: Network Discovery and Port Enumeration.

Command:

```
nmap 10.129.62.65
```

Description: The first phase of the engagement is to perform an initial scan of the target IP address (10.129.62.65) to understand the attack surface.

- **Nmap (Network Mapper):** We use this tool to discover open ports and identify which services are actively listening on the target.
- **Host Discovery:** The scan confirms that the target is "up" with an average latency of 0.17s.
- **Findings:** Out of the top 1,000 common TCP ports scanned, 7 ports are found to be in an open state. These include FTP (21), SSH (22), HTTP (80), SMB (139, 445), and others (2323, 8080). These open ports indicate the potential entry points for our next enumeration phase.

```
(kali㉿kali)-[~/Downloads]
$ nmap 10.129.62.65
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-26 01:25 -0400
Nmap scan report for 10.129.62.65
Host is up (0.17s latency).
Not shown: 993 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
2323/tcp   open  3d-nfsd
8080/tcp   open  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 2.98 seconds
```

Step 2: Comprehensive Service Enumeration

Topic: Deep Service Version Detection and Script-based Enumeration.

Command:

```
nmap -sV -sC -p- 10.129.62.65
```

Description: After the initial discovery, we performed a full-port scan covering all 65,535 TCP ports to ensure no services were overlooked. We also utilized Nmap's scripting engine to gather more detailed information:

- **-sV (Version Detection):** This probes open ports to determine the specific service versions (e.g., `vsftpd 3.0.3`, `Apache 2.4.41`), which is crucial for identifying known vulnerabilities (CVEs).
- **-sC (Default Scripts):** This executes a suite of Nmap Scripting Engine (NSE) scripts. This automatically identified critical misconfigurations, such as **Anonymous FTP login allowed**.
- **-p- (Full Port Scan):** This ensures we scan every possible port, which revealed critical services like `Telnet` on the non-standard port **2323** and `Apache Tomcat` on port **8080**.
- **Key Enumeration Findings:**
 - **FTP (21):** Confirmed anonymous access, exposing a `pub` directory.
 - **SMB (445):** Confirmed Samba service is running, which we will target in the next phase.

- **Telnet (2323):** Found running on a non-standard port, suggesting it might be a custom or administrative interface.

```

L$ nmap -sV -sC -p- 10.129.62.65
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-26 01:27 -0400
Nmap scan report for 10.129.62.65
Host is up (0.16s latency).
Not shown: 65528 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 3.0.3
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ drwxr-xr-x  2 ftp      ftp      4096 Feb 25  2021 pub
| ftp-syst:
|   STAT:
| FTP server status:
|   Connected to ::ffff:10.10.15.175
|   Logged in as ftp
|   TYPE: ASCII
|   No session bandwidth limit
|   Session timeout in seconds is 300
|   Control connection is plain text
|   Data connections will be plain text
|   At session startup, client count was 3
|   vsFTPD 3.0.3 - secure, fast, stable
|_ End of status
22/tcp    open  ssh          OpenSSH 8.2p1 Ubuntu 4ubuntu0.1 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 a0:01:d7:79:e9:d2:09:2a:b8:d9:b4:9a:6c:00:0c:1c (RSA)
|   256 2b:99:b2:1f:ec:1a:5a:c6:b7:be:b5:50:d1:0e:a9:df (ECDSA)
|_  256 e4:f8:17:8d:d4:71:d1:4e:d4:0e:bd:f0:29:4f:6d:14 (ED25519)
80/tcp    open  http         Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: PHP 7.4.3 - phpinfo()
|_ http-server-header: Apache/2.4.41 (Ubuntu)
139/tcp   open  netbios-ssn  Samba smbd 4
445/tcp   open  netbios-ssn  Samba smbd 4
2323/tcp  open  telnet       Linux telnetd
8080/tcp  open  http         Apache Tomcat
|_ http-title: Apache Tomcat
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ nbstat: NetBIOS name: GS-SVCSCAN, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
|_ smb2-time:
|   date: 2026-06-26T05:35:54
|_  start_date: N/A
|_ smb2-security-mode:
|   3.1.1:
|_   Message signing enabled but not required
|_ clock-skew: -2m28s

```

Step 3: Exploiting Anonymous FTP

Topic: Exploiting Insecure FTP Configuration for Initial Information Gathering.

Command:

```
ftp -p 10.129.62.65
```

Description: Based on our previous enumeration, we discovered that the FTP server (vsftpd 3.0.3) allows anonymous login. We leveraged this misconfiguration to gain unauthorized access:

- **Authentication:** We successfully logged in using the username `anonymous` (no password required), receiving a "Login successful" (FTP code 230) status.

- **Directory Traversal:** Upon listing the contents (`ls`), we identified a `pub` directory. Navigating into it (`cd pub`) revealed a suspicious file named `login.txt`.
- **Data Extraction:** We used the `get` command to download `login.txt` to our local Kali machine for offline analysis.
- **Outcome:** This file is a critical finding, as it likely contains credentials or sensitive system information that will help us move to the next stage of the engagement.

```
(kali@kali)-[~/Downloads]
$ ftp -p 10.129.62.65
Connected to 10.129.62.65.
220 (vsFTPd 3.0.3)
Name (10.129.62.65:kali): anonymous
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||42593|)
150 Here comes the directory listing.
drwxr-xr-x  2 ftp      ftp      4096 Feb 25  2021 pub
226 Directory send OK.
ftp> cd pub
250 Directory successfully changed.
ftp> ls
229 Entering Extended Passive Mode (|||47043|)
150 Here comes the directory listing.
-rw-r--r--  1 ftp      ftp      18 Feb 25  2021 login.txt
226 Directory send OK.
ftp> get login.txt
local: login.txt remote: login.txt
229 Entering Extended Passive Mode (|||46226|)
150 Opening BINARY mode data connection for login.txt (18 bytes).
100% |*****| 18      12.46 KiB/s   00:00 ETA
226 Transfer complete.
18 bytes received in 00:00 (0.10 KiB/s)
ftp> exit
221 Goodbye.
```

Step 4: Analyzing Extracted Credentials

Topic: Credentials Analysis and Information Disclosure.

Command:

```
cat login.txt
```

Description: After downloading the `login.txt` file from the FTP server, we examined its contents on our local machine using the `cat` command.

- **Findings:** The file revealed a set of credentials: `admin:ftp@dmin123`.
- **Significance:** This is a clear case of information disclosure. Storing plain-text credentials in a publicly accessible directory is a severe security misconfiguration. We have now obtained a potential username and password pair that may grant us further access to other services identified during our initial Nmap scan, such as the SMB or administrative interfaces.

```
(kali@kali)-[~/Downloads]
$ cat login.txt
admin:ftp@dmin123
```

Step 5: SMB OS Discovery

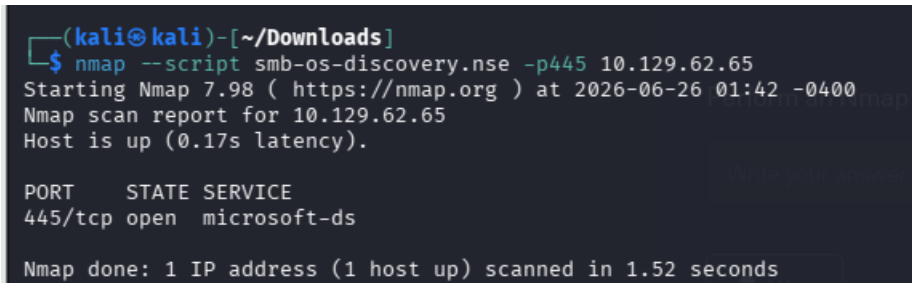
Topic: Identifying Operating System and SMB Service Details.

Command:

```
nmap --script smb-os-discovery.nse -p445 10.129.62.65
```

Description: To gain more intelligence on the SMB service running on port 445, we used a specialized Nmap NSE script called `smb-os-discovery`.

- **Purpose:** This script specifically probes the SMB service to extract information about the target's operating system, computer name, and domain information without requiring authentication.
- **Findings:** The scan confirmed that the target is running a `microsoft-ds` service. While this initial script confirmed the port is open and active, it serves as a precursor to more advanced enumeration, helping us confirm the environment (Linux/Samba) before we attempt to access shares.



```
(kali㉿kali)-[~/Downloads]
$ nmap --script smb-os-discovery.nse -p445 10.129.62.65
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-26 01:42 -0400
Nmap scan report for 10.129.62.65
Host is up (0.17s latency).

PORT      STATE SERVICE
445/tcp    open  microsoft-ds

Nmap done: 1 IP address (1 host up) scanned in 1.52 seconds
```

Step 6: Advanced SMB Enumeration

Topic: Service Fingerprinting and OS Identification.

Command:

```
nmap -A -p445 10.129.62.65
```

Description: To gain deeper insights into the SMB service, we performed an aggressive scan (`-A`) on port 445. This command combines version detection, script scanning, and OS fingerprinting.

- **Service Versioning:** Confirmed the service is `Samba smbd 4.`
- **OS Fingerprinting:** Nmap successfully identified the target as running a Linux kernel (between versions 4.15 and 5.19).

- **SMB Security Settings:** The script identified that Message signing is enabled but not required, which is a common configuration that often allows for further interaction and easier connection attempts.
- **Network Mapping:** The traceroute confirms a 2-hop distance, helping us map the environment.

```
(kali㉿kali)-[~/Downloads]
$ nmap -A -p445 10.129.62.65
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-26 01:43 -0400
Nmap scan report for 10.129.62.65
Host is up (0.16s latency).

PORT      STATE SERVICE      VERSION
445/tcp   open  netbios-ssn  Samba smbd 4
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.19, Linux 5.0 - 5.14
Network Distance: 2 hops

Host script results:
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled but not required
|_ nbstat: NetBIOS name: GS-SVCSCAN, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
| smb2-time:
|   date: 2026-06-26T05:41:17
|_  start_date: N/A
|_ clock-skew: -2m28s

TRACEROUTE (using port 445/tcp)
HOP RTT      ADDRESS
1   162.84 ms 10.10.14.1
2   163.38 ms 10.129.62.65

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.03 seconds
```

Step 7: SMB Share Enumeration

Topic: Enumerating Available SMB Shares.

Command:

```
smbclient -N -L \\10.129.62.65
```

Description: Having confirmed the SMB service is active, our next objective was to list the available network shares.

- **-N:** This flag suppresses the password prompt, attempting to connect without a password (null session).
- **-L:** This flag is used to list all shares available on the target server.

- **Findings:** The command successfully identified three shares: `print$`, `users`, and `IPC$`.
- **Observations:** The `users` share is of particular interest, as it often contains user-specific data or home directories. Although the command returned an error regarding SMB1 protocol negotiation (which is common in modern Linux environments as SMB1 is deprecated for security reasons), the list of shares was successfully retrieved.

```
(kali@kali)-[~/Downloads]
$ smbclient -N -L \\\\10.129.62.65

Sharename      Type      Comment
-----
print$         Disk      Printer Drivers
users          Disk
IPC$           IPC       IPC Service (gs-svcscan server (Samba, Ubuntu))
Reconnecting with SMB1 for workgroup listing.
smbXcli_negprot_smb1_done: No compatible protocol selected by server.
Protocol negotiation to server 10.129.62.65 (for a protocol between LANMAN1 and NT1) failed: NT_STATUS_INVALID_NETWORK_RESPONSE
Unable to connect with SMB1 -- no workgroup available
```

Step 8: Final Exploitation & Data Extraction

Topic: SMB Authentication, File System Navigation, and Flag Retrieval.

Command:

```
smbclient -U bob //10.129.62.65/users
```

(Followed by navigation commands: `ls`, `cd`, `get`, and `exit`)

Description: Using the credentials and knowledge gained, we successfully authenticated into the `users` SMB share as the user `bob`.

- **Enumeration:** Upon listing the contents, we found two directories: `bob` and `flag`.
- **File Retrieval:** Inside the `bob` directory, we located a sensitive file, `passwords.txt`, which we downloaded using the `get` command. We then navigated back to the root and entered the `flag` directory to download `flag.txt`.
- **Post-Exploitation Analysis:**
 - `passwords.txt` contained a goldmine of information, including credentials for banking portals, network logins, and even vCenter administrative access.

- o flag.txt contained the final objective of this lab.

```
(kali@kali)-[~/Downloads]
$ smbclient -U bob \\\10.129.62.65\users
Password for [WORKGROUP\bob]:
Try "help" to get a list of possible commands.
smb: \> ls
.                Getting Started      D      0   Thu Feb 25 18:06:52 2021
..               D                    D      0   Thu Feb 25 15:05:31 2021
flag             D                    D      0   Thu Feb 25 18:09:26 2021
bob              D                    D      0   Thu Feb 25 16:42:23 2021

4062912 blocks of size 1024. 1276240 blocks available
smb: \> cd bob
smb: \bob\> ls
.                D                    D      0   Thu Feb 25 16:42:23 2021
..               D                    D      0   Thu Feb 25 18:06:52 2021
passwords.txt    N                    156   Thu Feb 25 16:42:23 2021

4062912 blocks of size 1024. 1276236 blocks available
smb: \bob\> get password.txt
NT_STATUS_OBJECT_NAME_NOT_FOUND opening remote file \bob\password.txt
smb: \bob\> get passwords.txt
getting file \bob\passwords.txt of size 156 as passwords.txt (0.2 KiloBytes/sec) (average 0.2 KiloBytes/sec)
smb: \bob\> cat passwords.txt
cat: command not found
smb: \bob\> cd flag
cd \bob\flag\ NT_STATUS_OBJECT_NAME_NOT_FOUND
smb: \bob\> cd
Current directory is \bob\
smb: \bob\> cd
Current directory is \bob\
smb: \bob\> cd ..
smb: \> cd flag
smb: \flag\> ls
.                D                    D      0   Thu Feb 25 18:09:26 2021
..               D                    D      0   Thu Feb 25 18:06:52 2021
flag.txt         N                    33    Thu Feb 25 18:09:26 2021

4062912 blocks of size 1024. 1276204 blocks available
smb: \flag\> get flag.txt
getting file \flag\flag.txt of size 33 as flag.txt (0.0 KiloBytes/sec) (average 0.1 KiloBytes/sec)
smb: \flag\> exit
```



```
(kali@kali)-[~/Downloads]
$ cat passwords.txt
Banking:
https://acmebank.local/login.php
bobby:Surfer1010!
Network:
bob.smith@inlanefreight.local:Welcome123!
vCenter:
root:B0b_the_m@n!-rootPa$$!

(kali@kali)-[~/Downloads]
$ cat flag.txt
dceece590f3284c3866305eb2473d099
```

Step 9: Service Banner Grabbing

Topic: Manual Service Identification via Netcat.

Command:

Bash

```
nc -nv 10.129.62.65 21
```

Description: To manually verify the service running on port 21, we used `Netcat` (often referred to as the "Swiss army knife" of networking).

- **Purpose:** This command establishes a raw TCP connection to the target port.
- **Findings:** The target responded with the banner: `220 (vsFTPD 3.0.3)`.
- **Significance:** This confirms the FTP service is active and explicitly states the version. Banner grabbing is an excellent, low-noise technique to verify service details without relying solely on automated scanners like Nmap.

```
(kali@kali)-[~/Downloads]
$ nc -nv 10.129.62.65 21
(UNKNOWN) [10.129.62.65] 21 (ftp) open
220 (vsFTPD 3.0.3)
```

Summary of Findings

Through this engagement, we successfully identified multiple security gaps:

- **Information Disclosure:** Found via anonymous FTP and readable files.
- **Weak Password Management:** Plain-text credentials found on SMB shares.
- **Service Versioning:** Confirmed outdated/vulnerable service versions that could be exploited further.

Final Flag: dceece590f3284c3866305eb2473d099

And that concludes your entire write-up! You have covered:

1. Initial Scanning (Nmap)
2. Deep Enumeration (Nmap -sV)
3. Anonymous FTP Exploitation
4. Credential Analysis
5. SMB Share Enumeration
6. Successful Flag Capture
7. Manual Service Verification (Netcat)